

1) D/B var, let & const :

	var	let	const
Feature			
Scope	Function Scope	Block Scope	Block Scope
Re-declare	yes	No	No
Re-assign	yes	yes	No
Hoisted	yes	yes(TDZ)	yes(TDZ)

2) Hoisting:

Hoisting is JavaScript's behavior of moving variable and function declarations to the top of their scope during the memory creation phase.

Example :

```
console.log(a);
```

```
var a = 10;
```

o/p:

undefined

3) TDZ :

TDZ is the period between entering a scope and initializing a let or const variable.

Example:

```
console.log (name);  
let name = "Bharani";
```

O/P:

Reference Error

4) Execution Context :

Execution Context is the environment in which JavaScript code executes

Phases

1) Memory Creation Phase:

→ Variables get memory.

→ Functions get complete memory.

2) Execution Phase:

→ Code executes line by line

→ Values are assigned.

Example:

```
var x = 10;
```

```
function test () {}
```

5) Scope:

Scope determines where variables can be accessed.

Global scope:

```
let name = "Bharani";
```

* Accessible everywhere.

Function Scope:

```
const functionScope () => {
```

```
  return a
```

```
}
```

```
functionScope()
```

* Accessible only inside function

Block Scope:

```
if (true) {
```

```
  let city = "Chennai";
```

```
}
```

* Accessible only inside block.

6) Lexical Scope ? Scope Chain

Lexical Scope means inner functions can access variables from outer functions.

Scope chain is the process of searching variables from current scope to outer scopes.

7) Closure:

A closure is a function that remembers variables from its outer scope even after the outer function has finished execution.

Example:

```
function counter() {  
  let count = 0;  
  return function() {  
    count++;  
    return count;  
  }  
}  
  
const increment = counter();  
console.log(increment());  
console.log(increment());
```

8) Closure preserve variables:

Because JavaScript stores references to the outer function's variables in memory as long as the inner function is using them.

9) Callback Function :

A Callback Function is a function passed as an argument to another function.

Example:

```
const ExampleFunction = (callback) => {
  callback();
};

ExampleFunction(() => {
  console.log("welcome");
});
```

Callbacks used in JavaScript :

- * Asynchronous programming
- * Event handling
- * Reusable coding

10) Callback Injection :

Callback Injection means dynamically passing different callback functions into the same function.

Example:

```
const processPayment = (cb) => {
  cb();
};
```

```
const qpay = () => console.log("qpay");  
const paytm = () => console.log("paytm");  
processPayment(qpay);  
processPayment(paytm);
```

11) Callback vs Closure :

callback

- * Function passed as argument
- * Used for execution later
- * Focus on behaviour

closure

- * Function remembers outer variables.
- * Used for data persistence
- * Focus on memory

12) Rest parameter vs spread operator

Rest parameter:

Collects multiple values into one array

```
function total (...nums) {  
  console.log(nums);  
}
```

Spread operator:

Expands array/object values

```
const arr = [1, 2, 3];  
console.log(...arr);
```

13) Return vs console.log

return:

Returns a value from a function

```
const add = (a,b) => {  
  return a + b;  
};  
const result = add(10,20);  
console.log(result);
```

console.log()

- * Prints data to the console for debugging
- * Does not return anything
- * Function continues executing after logging

14) Truthy and Falsy values:

JavaScript automatically converts values into true or false during conditional checks. This process is called Type coercion.

Falsy values:

false, 0, -0, null, undefined, NaN

Truthy values

everything else is truthy.

15) D/R Shallow copy and Deep copy

Shallow copy:

Copies only the first level.

```
const user = {  
  name: "Bharani"  
  address: {  
    city: "chennai"  
  }  
};
```



```
const copy = { ...user };  
copy.address.city = "Madurai";  
console.log (user.address.city);
```

Deep copy:

- * Creates completely independent copies:
- * Deep copy creates entirely new copies of nested objects

16) D/B map(), filter() and reduce():

map()

The map() method creates a new array by transforming every element of the original array according to a specified function.

filter()

The filter() method creates a new array containing only the elements that satisfy a specified condition.

reduce()

The reduce() method processes all elements of an array and reduces them to a single value such as a sum, average, object or string.

17) D/B `forEach()` and `map()`?

`forEach()`

`forEach()` executes a function for each array element and is mainly used when performing actions such as printing values or updating external variables.

`map()`

executes a function for each element and returns a new array containing the transformed values.

18) D/B `find()` and `filter()`:

`find()`

The `find()` method returns the first element that satisfy a specified condition and continues once a match is found, it stops searching.

`filter()`

The `filter()` method returns all elements that satisfy a specified condition and continues checking every element in the array.

19) D/B `for...of`, `for...in` & `forEach()`

`for...of`

The `for...of` loop is used to iterate over the values of iterable objects such as arrays, strings, maps, and sets.

for...in

The for...in loop is used to iterate over the keys or property names of an object.

forEach()

forEach() is an array method used to execute a function for each array element.

20) Object Destructuring:

Object destructuring is an ES6 feature that allows properties of an object to be extracted and assigned directly to variables in a concise manner.

Enhanced Object Literals:

Enhanced object literals are ES6 features that simplify object creation by allowing property shorthand, method shorthand.

React applications frequently work with props, state, and API data, which are usually stored as objects.